Email Filter

Strategy-pattern system for automated email categorization

Introduction

Smart Email Filtering System addresses inefficient manual sorting by applying the **Strategy Pattern** to classify messages automatically.

The solution reduces manual effort and improves accuracy through modular, swappable strategies, enabling consistent categorization (Spam / Work / Social) and robust error handling for all users.

01

System Design

Architecture Layers (UI, Manager, Strategy, Model)

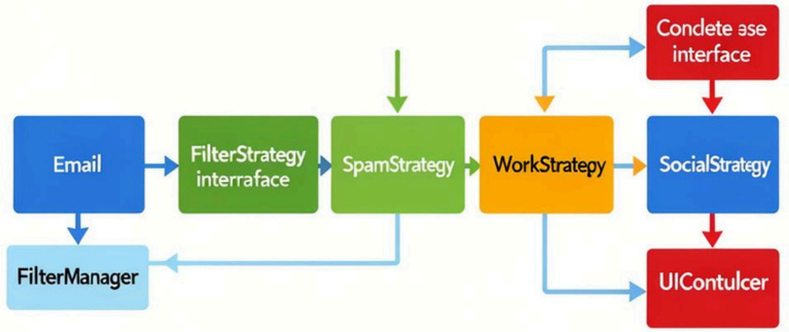
UI captures inputs and displays results. *Manager* orchestrates flow and applies policies.

Strategy layer contains classification implementations; *Model* stores email data and labels. This separation ensures clear responsibilities and supports horizontal scalability and maintainability.

Key Classes & Structure

Core classes: *Email*, *FilterStrategy* (interface), *SpamStrategy*, *WorkStrategy*, *SocialStrategy*, *FilterManager*, *UIController*.

FilterStrategy defines classification contract; concrete strategies implement keywords and rules. Design enables **extensible** strategies, simple testing, and straightforward error handling.



Data Flow & Scalability

Emails flow: *UIController* - *FilterManager* - selected *FilterStrategy* - *Model* for persistence.

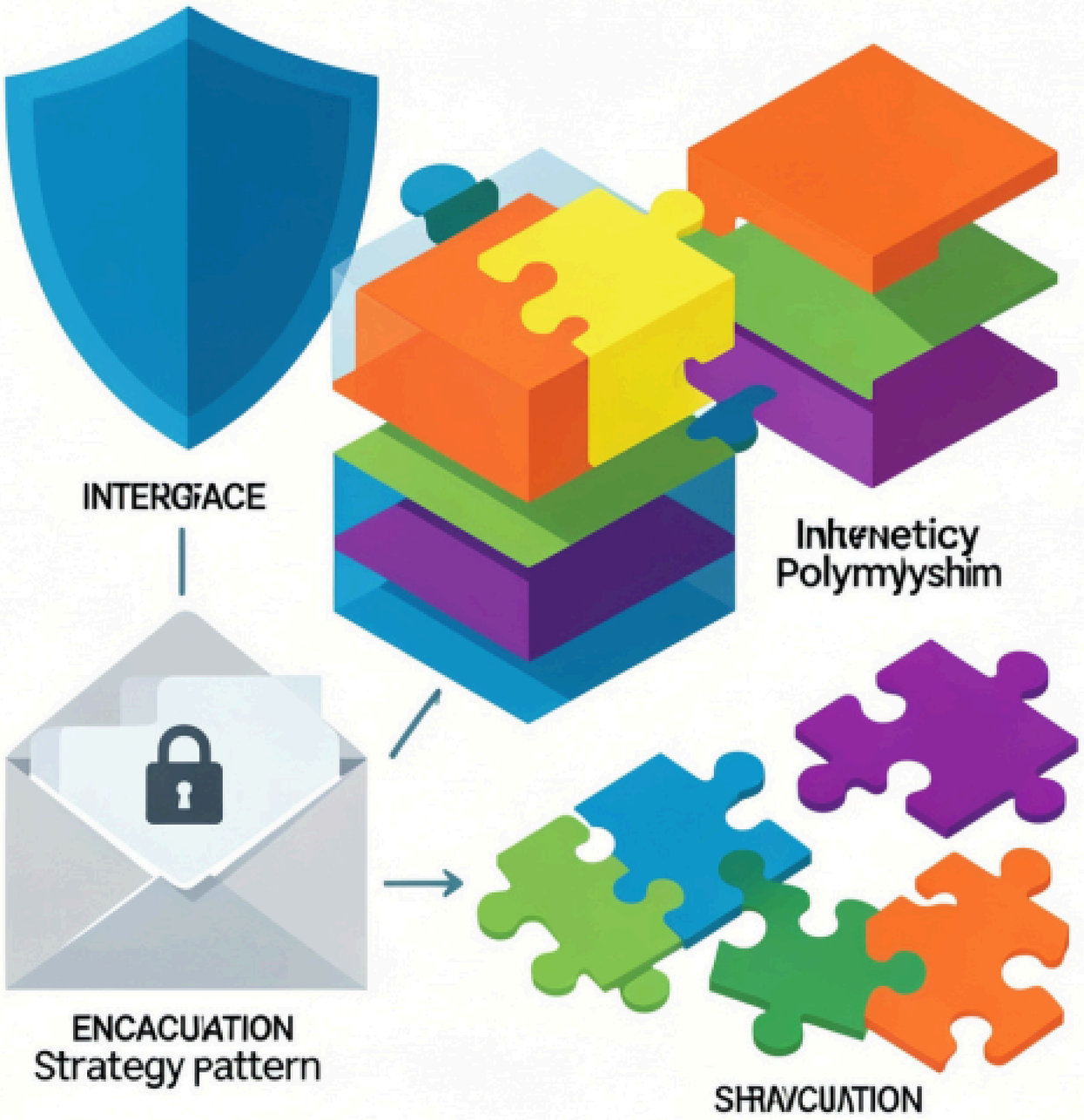
Design supports horizontal scaling via stateless managers and distributed storage. **Asynchronous processing** and batch classification enable high throughput and low latency.

OOP Concepts & Strategy Pattern

Abstraction: FilterStrategy interface hides implementation details.

Inheritance Polymorphism: Concrete strategies extend the contract and are interchangeable at runtime.

Encapsulation secures email state; Strategy Pattern enables clean, testable extensions.



Keyword Classification & Tech Highlights

Keyword sets per strategy (e.g., "invoice", "meeting" for Work) drive rule-based classification.

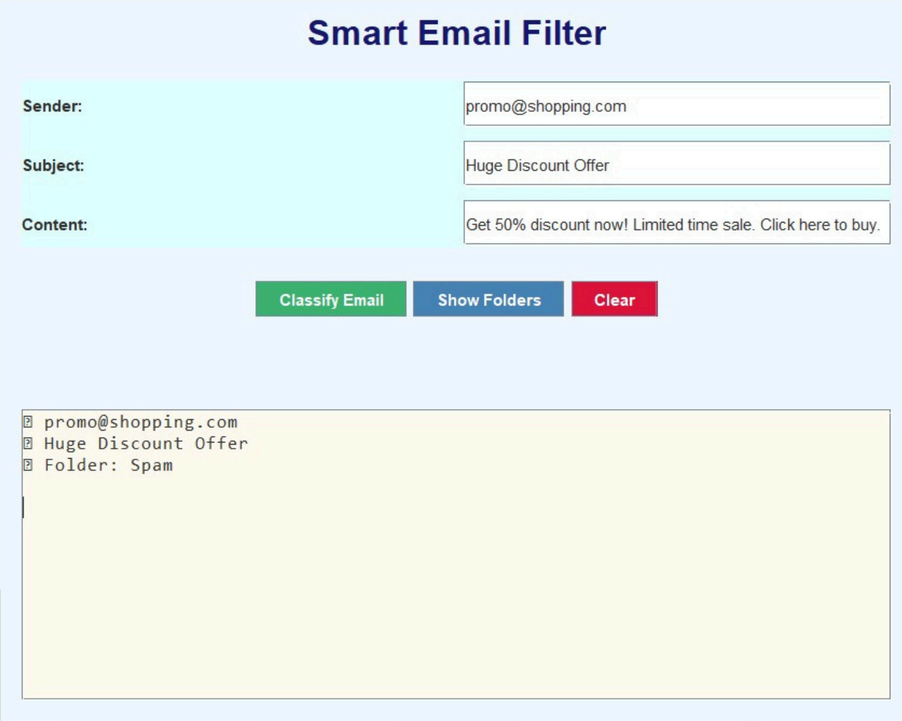
Hybrid approach allows fallback to ML models for ambiguous cases.

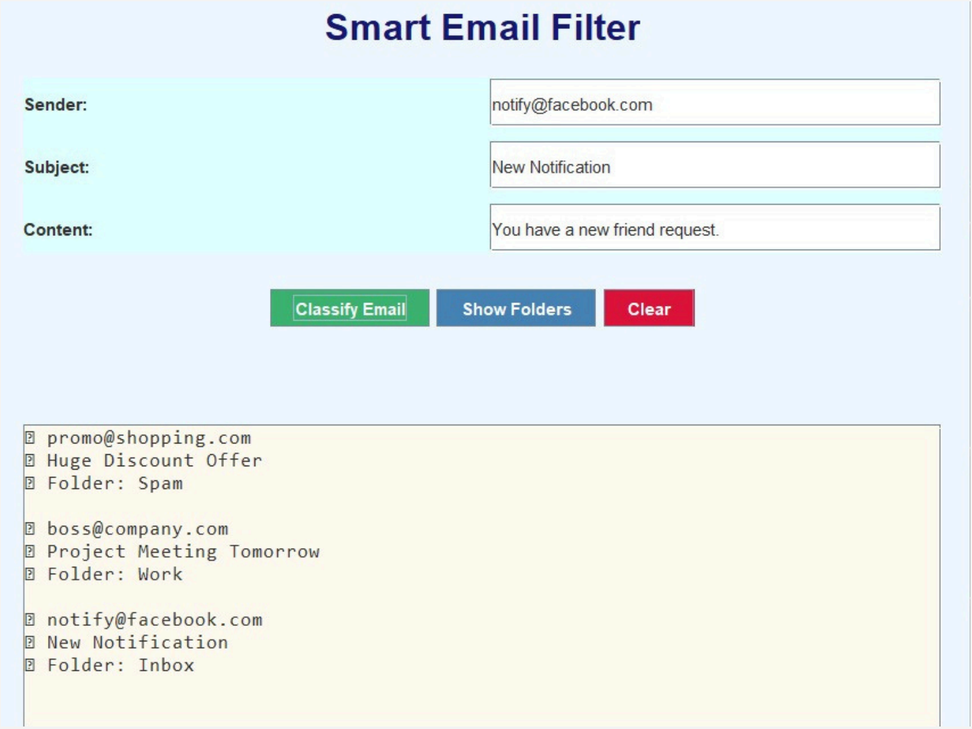
Modular strategies simplify adding new categories and tuning rules.

Live Demo: Inputs, Categorization, Error Handling

Demo shows raw input routed to manager, strategy selection, and label assignment (Spam / Work / Social).

Errors (parsing, missing fields) produce structured logs and user-friendly messages; retries and quarantines handle failures gracefully.

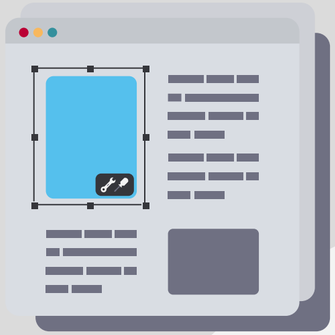




Conclusions

The Strategy Pattern yields a modular, scalable filter system with clear responsibilities and easy extensibility.

Adopting keyword strategies with ML fallbacks provides accuracy and operational resilience while simplifying maintenance and future growth.



Thanks!